Week 14 Technical Reflection

Student: Laurence Matthew B Manuel **Section:** BSIT 3A **Project:** Project 29 Thesis Consultation Reminder System
Basis: Verified Week 14 implementation and evidence completed 8 November 2026 Asia/Manila

Decision with the greatest effect

The decision with the greatest effect on security was to use authenticated encryption in addition to HTTPS. HTTPS protects the browser connection, credentials, session token, and endpoint identity, but it does not by itself satisfy the controlled payload-exchange exercise or provide application-level freshness. AES-256-GCM allowed the receiver to detect changes to both the ciphertext and the authenticated sender, recipient, message, and timestamp metadata. The altered-ciphertext test returned HTTP 422 without producing plaintext or a replay receipt, while the original envelope was accepted once and its replay returned HTTP 409. These results confirmed that encryption, integrity protection, expiry, authorization, and replay storage solve separate parts of the problem and must be used together.

The live CORS failure also changed the way I evaluate secure transport. A valid certificate and correct API were not enough when the gateway failed to forward the required preflight metadata. The browser blocked the request and displayed a safe network error. Forwarding only the two required preflight headers restored the path without weakening the origin allowlist. This showed that reliable secure exchange depends on correct boundary configuration as well as cryptographic choices.

Most difficult case to reproduce

An authentically encrypted but expired envelope was the most difficult adverse case to reproduce in the live five-minute browser flow. Waiting for every browser run would make the test slow and unreliable, while shortening the configured lifetime only for evidence could test behavior different from the intended design. The backend clock-controlled test created a valid envelope, advanced time, and verified rejection without changing the normal five-minute policy.

I would improve this test by injecting a small clock interface into `SecureExchange`. Production would use the system clock, while tests could supply exact issue, expiry, and future-skew times. This would isolate the time dependency and avoid framework-wide time travel affecting unrelated authentication or database behavior.

Artifact I can explain and modify

The artifact I will use for independent explanation is `backend/app/Services/SecureExchange.php`, supported by `backend/tests/Feature/SecureExchangeTest.php`. I can trace how the service loads a protected key, generates a random nonce, builds canonical additional authenticated data, encrypts with AES-256-GCM, validates encoding and timestamps, authenticates before decoding plaintext, and returns safe failures. A bounded modification I can demonstrate is changing the permitted lifetime through server configuration and updating the associated expiry test without placing key material in Flutter or source control.

The tests are important to the explanation because they distinguish a cryptographic success from an authorized application success. A valid tag is still insufficient when the current user has the wrong role, is not the named recipient, is not assigned to the consultation, or submits a message that already has a receipt.

What should be verified next

Before broader use, an authorized maintainer should replace the local evidence gateway and CA with a maintained deployment proxy and institution-approved certificate service, move the AES key to an approved protected secret store, define session revocation and replay-receipt retention, monitor clock drift and security events, and test backup and rotation under assigned operational ownership. A manual accessibility review and an authorized penetration review should also be completed.

These items are outside the current evidence because they require deployment infrastructure, governance decisions, operational owners, approved participants, or instructor authorization beyond the bounded local Week 14 build. The existing results support a controlled assessment prototype, not production readiness or institutional security approval.

Reflection conclusion

The Week 14 work demonstrates that selecting an algorithm is only one part of secure integration. The defensible result comes from combining an established authenticated-encryption library with HTTPS, externalized keys, minimized data, server-side authorization, bounded validity, durable replay rejection, safe logging, controlled failure evidence, and an honest residual-risk record. The most useful evidence was not only the successful exchange but also the failures that remained safe and the regressions that proved the fixes did not weaken existing behavior.